

Datos del alumno

Nombre: Manuel Ramírez Galván

Numero de actividad: 3

Fecha: 05/02/2025

✓ Actividad Transformaciones, imagenes a color y modelos de color

Objetivos: Realizar transformaciones en imagenes, como tambien manipular imágenes a color para convertirlas a diferentes modelos de color.

✓ Ejercicio 1: Importar bibliotecas necesarias

```
import numpy as np
import cv2
import matplotlib.pyplot as plt
```

```
from google.colab import drive
drive.mount('/content/drive')
```

 Mounted at /content/drive

✓ Ejercicio 2: Operaciones de Suma y Resta en Imágenes

Objetivo: Aprender a realizar operaciones de suma y resta en imágenes utilizando OpenCV y visualizar los resultados con Matplotlib.

Pasos:

1. Carga de imágenes usa `cv2.imread()` para cargar ambas imágenes en color. Asegúrate de que la ruta sea válida y que los archivos existan.
2. Ajuste de tamaño, si las imágenes tienen diferentes dimensiones, redimENSIONALAS con `cv2.resize()` para que tengan el mismo tamaño.
3. Realiza la suma de las dos imágenes utilizando `cv2.add()`.
4. Realiza la resta de las dos imágenes utilizando `cv2.subtract()`.
5. Muestra las imágenes originales, la imagen resultante de la suma y la imagen resultante de la resta utilizando `plt.imshow()`.

```
img1 = cv2.imread("/content/drive/MyDrive/Colab Notebooks/1_3_Transformaciones_Color_Modelos/data/loro.png", cv2.IMREAD_COLOR)
img1 = cv2.cvtColor(img1, cv2.COLOR_BGR2RGB)
img2 = cv2.imread("/content/drive/MyDrive/Colab Notebooks/1_3_Transformaciones_Color_Modelos/data/raton.png", cv2.IMREAD_COLOR)
img2 = cv2.cvtColor(img2, cv2.COLOR_BGR2RGB)
```

```
img2 = cv2.resize(img2, (img1.shape[1], img1.shape[0]))
```

```
img_sum = cv2.add(img1, img2)
img_res = cv2.subtract(img1, img2)
```

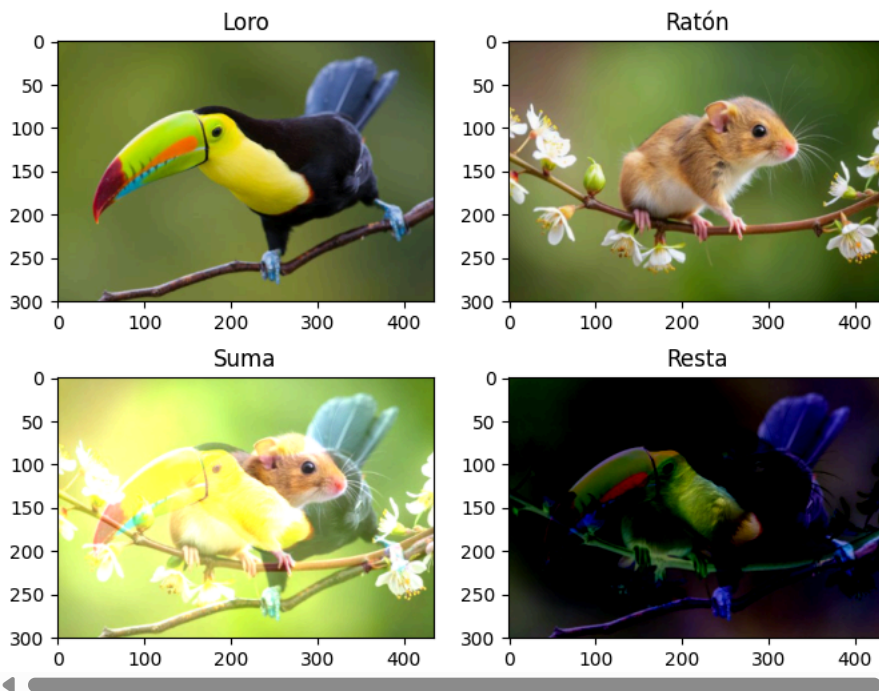
```
fig, axs = plt.subplots(2, 2, figsize=(8, 6))
axs[0,0].imshow(img1)
axs[0,0].set_title("Loro")
```

```
axs[0,1].imshow(img2)
axs[0,1].set_title("Ratón")
```

```
axs[1,0].imshow(img_sum)
axs[1,0].set_title("Suma")
```

```
axs[1,1].imshow(img_res)
axs[1,1].set_title("Resta")
```

↩ Text(0.5, 1.0, 'Resta')



✓ Ejercicio 3: Cargar y visualizar una imagen a color

Objetivo: Aprender a cargar una imagen a color y visualizarla con Matplotlib.

Pasos:

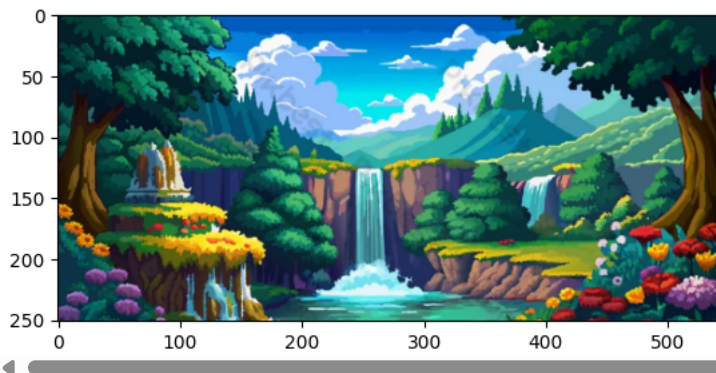
1. Cambia la variable `image_path` para que apunte al archivo de imagen que quieras que esta en `data`. Asegúrate de que la ruta sea válida y que el archivo exista.
2. Usa la función `cv2.imread()` con el parámetro `cv2.IMREAD_COLOR` para cargar la imagen a colores. Asegurate que la imagen si sea a color.
3. Muestra la imagen con `plt.imshow()`.

```
img3 = cv2.imread("/content/drive/MyDrive/Colab Notebooks/1_3_Transformaciones_Color_Modelos/data/bosque.png", cv2.IMREAD_COLOR)
img3 = cv2.cvtColor(img3, cv2.IMREAD_COLOR)
```

```
img_rgb = cv2.cvtColor(img3, cv2.COLOR_BGR2RGB)
```

```
plt.imshow(img_rgb)
```

↩ <matplotlib.image.AxesImage at 0x77fc9d5f0950>



✓ Ejercicio 4: Conversión entre modelos de color

Objetivo: Convertir una imagen entre modelos de color y observar los resultados.

Pasos:

1. Convierte la imagen al modelo HSV.
2. Convierte la imagen al modelo CMY.
3. Muestra la imagen con `plt.imshow()`.

```
img_hsv = cv2.cvtColor(img_rgb, cv2.COLOR_BGR2HSV)
```

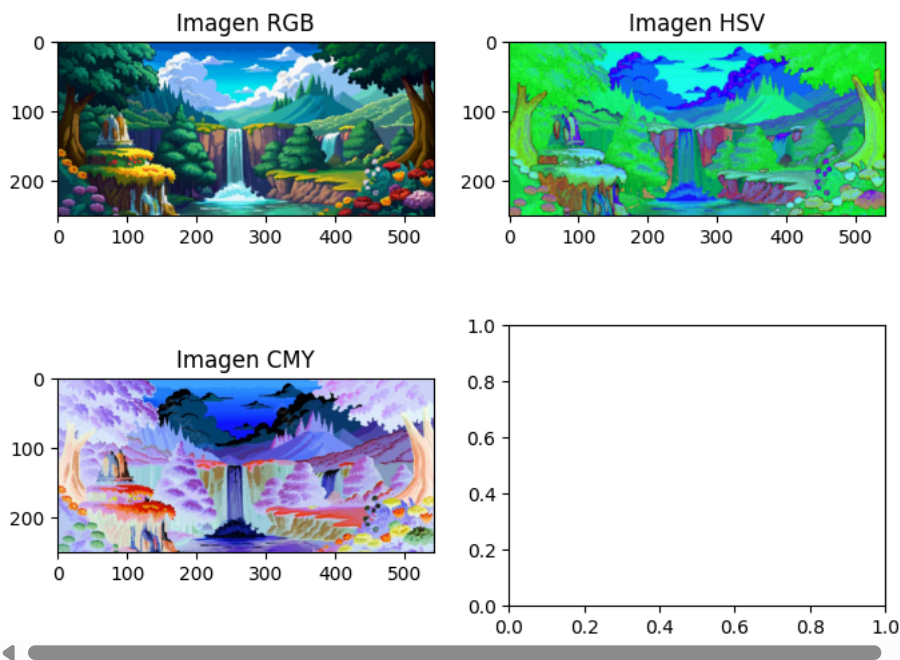
```
img_cmy = 1 - (img3 / 255)
```

```
fig, axs = plt.subplots(2, 2, figsize=(8, 6))
axs[0,0].imshow(img_rgb)
axs[0,0].set_title("Imagen RGB")
```

```
axs[0,1].imshow(img_hsv)
axs[0,1].set_title("Imagen HSV")
```

```
axs[1,0].imshow(img_cmy)
axs[1,0].set_title("Imagen CMY")
```

```
Text(0.5, 1.0, 'Imagen CMY')
```



✓ Ejercicio 5: Separación de canales de color

Objetivo: Aprender a separar los canales de las imágenes a color para después poder realizar modificaciones por separado.

Pasos:

1. Usa la función `c2.split()`, para separar los canales de la imagen.
2. Usa `plt.imshow()` y `plt.subplot()` para mostrar las imagen resultante a color.
3. Vuelve a unir las imagenes usando `c2.merge()`.
4. Usa `plt.imshow()` para mostrar las imagen original a color.

```
r, g, b = cv2.split(img_rgb)
```

```
fig, axs = plt.subplots(2, 2, figsize=(8, 6))
axs[0,0].imshow(r, cmap='Reds')
axs[0,0].set_title("Canal Rojo")
```

```
axs[0,1].imshow(g, cmap='Greens')
axs[0,1].set_title("Canal Verde")
```

```
axs[1,0].imshow(b, cmap='Blues')
axs[1,0].set_title("Canal Azul")
```

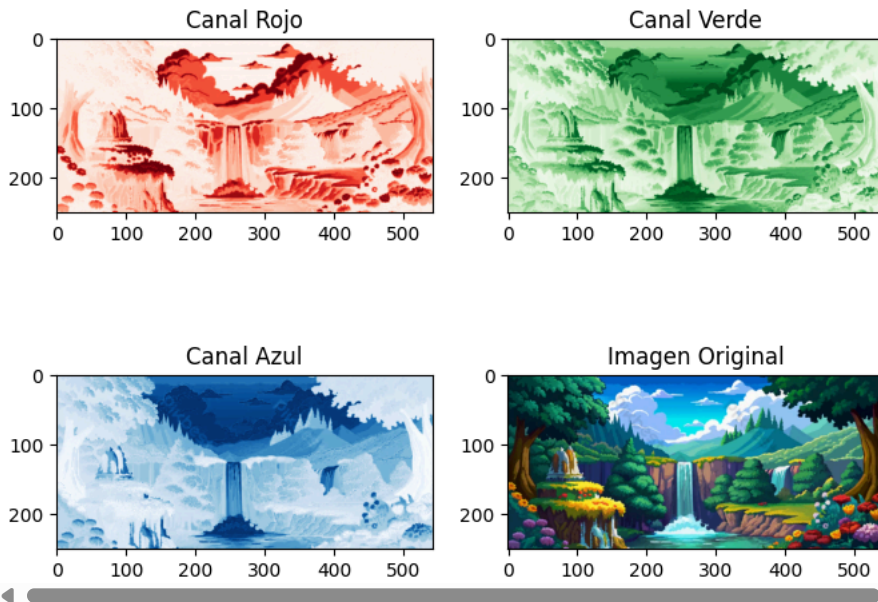
```
img_merge = cv2.merge([r, g, b])
```

```

axs[1,1].imshow(img_merge)
axs[1,1].set_title("Imagen Original")

Text(0.5, 1.0, 'Imagen Original')

```



✓ Ejercicio 6: Modificar tono, saturación y brillo

Objetivo: Aprender a analizar las dimensiones de una imagen, acceder al valor de un píxel específico, modificarlo, y observar el resultado.

Pasos:

1. Convierte a HSV: Convierte la imagen a HSV con `cv2.cvtColor()` para manipular directamente el tono, la saturación y el brillo.
2. Modifica el Tono (Hue): Suma un valor al canal de tono y asegurate de que el rango esté entre 0-179 (rango aceptado por OpenCV para el tono).
3. Modifica la Saturación: escala el canal de saturación por un factor multiplicativo y usa `np.clip()` para limitar los valores entre 0-255.
4. Modificar el Brillo (Value): Incrementa el canal de brillo por un valor específico, usa `np.clip()` para asegurar que los valores estén en el rango 0-255.
5. Usa `cv2.merge()` para reconstruir la imagen HSV modificada. y convierte de nuevo a RGB con `cv2.cvtColor()` para visualizar los cambios.
6. Usa `plt.imshow()` para mostrar todos los resultados.

```

img_rgb = cv2.cvtColor(img_rgb, cv2.COLOR_BGR2RGB)
img_hsv = cv2.cvtColor(img_rgb, cv2.COLOR_RGB2HSV)

h, s, v = cv2.split(img_hsv)

plt.figure(figsize=(12, 6))
plt.subplot(2, 2, 1)
plt.imshow(img_rgb)
plt.title("Imagen Original")

hue_shift = 170
h_mod = (h + hue_shift) % 180
img_hue_mod = cv2.merge([h_mod, s, v])
img_hue_mod_brg = cv2.cvtColor(img_hue_mod, cv2.COLOR_HSV2RGB)

plt.subplot(2, 2, 2)
plt.imshow(img_hue_mod_brg)
plt.title(f"Tono modificado(+{hue_shift})")

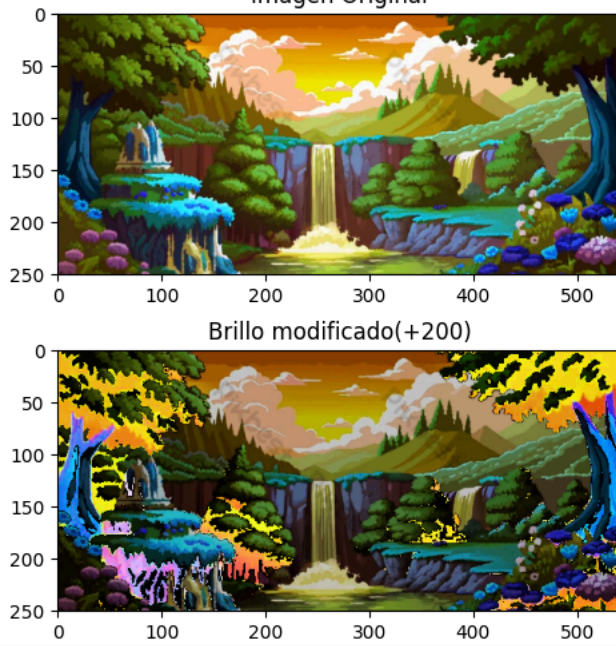
brighness_shift = 200
v_mod = np.clip(v + brighness_shift, 0, 255).astype(np.uint8)
img_brighness_mod = cv2.merge([h, s, v_mod])
img_brighness_mod_bgr = cv2.cvtColor(img_brighness_mod, cv2.COLOR_HSV2RGB)

plt.subplot(2, 2, 3)
plt.imshow(img_brighness_mod_bgr)
plt.title(f"Brillo modificado(+{brighness_shift})")

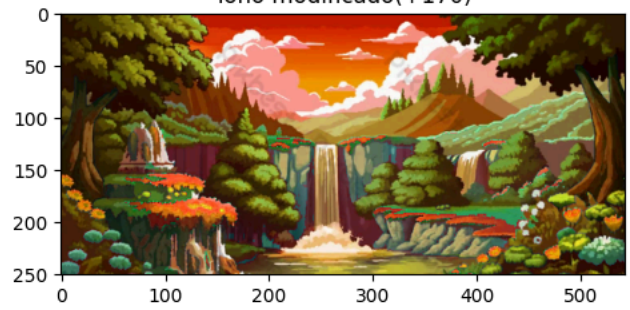
```

 `Text(0.5, 1.0, 'Brillo modificado(+200)')`

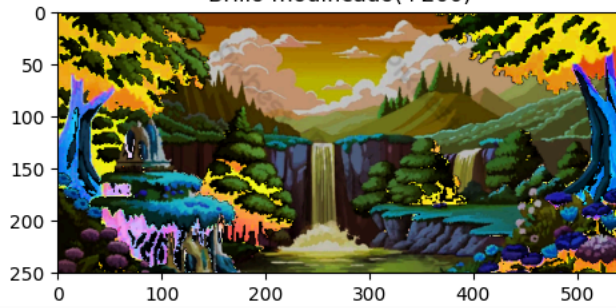
Imagen Original



Tono modificado(+170)



Brillo modificado(+200)



Comienza a programar o [generar](#) con IA.

No fue posible conectarse al servicio de reCAPTCHA. Comprueba tu conexión a Internet y vuelve a cargar la página para obtener un desafío de reCAPTCHA.